DIFF-TRO: A Self-supervised Diffusion Framework for IoT Topology Robustness Optimization

Jiqiao Li, *Student Member, IEEE,* Ning Chen, *Member, IEEE,* Xiaobo Zhou, *Senior Member, IEEE,* Tie Qiu, *Senior Member, IEEE*

***Abstract*—The robustness of Internet of Things (IoT) topologies measures a network's tolerance to node removals, which is critical for stable network connectivity. Existing robustness optimization methods mainly rely on search-based heuristics, which suffer from path dependence and often converge to suboptimal topologies. To address this limitation, we propose DIFF-TRO, a self-supervised diffusion framework for IoT topology robustness optimization. DIFF-TRO learns robustness-related structural priors directly through controlled noise injection and denoising in a spatio-temporal latent space, thereby mitigating path dependence. To translate latent representations into feasible and robust topologies, DIFF-TRO incorporates a constraint-aware decoding strategy that explicitly enforces geometric and connectivity constraints. In addition, a closed-loop refinement strategy progressively enhances robustness without relying on labeled data. Extensive experiments demonstrate that DIFF-TRO consistently outperforms state-of-the-art heuristic and reinforcement-learning baselines across diverse IoT network configurations.**

***Index Terms*—Internet of Things, Network Topology, Robustness Optimization, Graph Diffusion Model.**

I. Introduction

THE Internet of Things (IoT) has emerged as a fundamental infrastructure for modern intelligent systems, enabling ubiquitous connectivity among billions of devices across domains such as industrial automation [1], [2], intelligent transportation [3], healthcare [4], [5], and smart cities [6], supported by the evolution of 5G/6G communications [7] and intelligent hardware [8]. As deployments scale, node failures caused by hardware malfunctions, link interference, or cyberattacks [9] may propagate, trigger cascading effects, and collapse network connectivity, severely degrading the Quality of Service (QoS) [10]. Maintaining high connectivity under node removals [11] therefore becomes a central design objective, which we refer to as the IoT topology robustness optimization problem.

Existing studies on topology robustness optimization primarily focus on search-based heuristic algorithms that iteratively modify edges under connectivity constraints. Such methods include heuristic approaches [12], [13] and reinforcement learning (RL)–based variants [14], [15], [16], which rely on sequential local edge modifications and repeated robustness evaluations. While these approaches can improve robustness in medium-scale settings, their inherently sequential local edge modifications induce strong path dependence: early rewiring decisions restrict subsequent choices, progressively narrowing the feasible search region [17]. As locally biased decisions accumulate, large portions of the solution space—including many potentially high robustness configurations—become effectively unreachable. This structural limitation is difficult to fully alleviate by more elaborate heuristics or increased computational effort, causing search-based methods to consistently converge to structurally biased and suboptimal topologies.

Beyond search-based approaches, data-driven methods have been explored to mitigate path dependence by learning robust connectivity patterns directly from labeled data. Early studies typically adopt supervised learning paradigms, where global edge distributions are learned from labeled topology pairs generated offline [18]. Such labels are obtained by first optimizing an initial topology using heuristic algorithms and then pairing the optimized topology with its original counterpart. By learning from these topology pairs, data-driven models can generate candidate topologies in parallel rather than through sequential local edge modifications, thereby reducing the accumulation of path dependence during optimization. However, despite this conceptual advantage, data-driven approaches remain difficult to deploy in IoT robustness optimization due to two fundamental challenges: (*i*) supervised models rely on large sets of labeled robust topologies that are difficult to obtain in real IoT deployments because of privacy constraints, device heterogeneity, and dynamic operating conditions [19]; and (*ii*) existing frameworks lack decoding mechanisms tailored to IoT-specific feasibility and connectivity constraints, often leading to infeasible or structurally suboptimal topologies.

To overcome the limitations of both search-based heuristics and supervised data-driven approaches, this paper proposes DIFF-TRO, a self-supervised diffusion framework for IoT topology robustness optimization. DIFF-TRO first introduces a spatio-temporal topology embedding coupled with a noise-injection diffusion process that maps node positions, adjacency relations, and denoising timesteps into a continuous latent space. Within this space, controlled perturbations allow the model to learn robustness-related structural priors directly from unlabeled topologies, thereby reducing reliance on labeled datasets and mitigating path dependence via distributional sampling rather than sequential edge rewiring. To translate denoised latent representations into feasible IoT

This work is supported by National Science Fund for Distinguished Young Scholars of China (No. 62325208), Joint Funds of National Natural Science Foundation of China (No. U25B2043 and U25B2041), National Natural Science Foundation of China (No. 62272339 and 62572342). (Corresponding author: Tie Qiu).

Jiqiao Li and Xiaobo Zhou are with the School of Computer Science and Technology, College of Intelligence and Computing, Tianjin University, Tianjin 300350, China (e-mail: lijiqiao@tju.edu.cn; xiaobo.zhou@tju.edu.cn).

Ning Chen is with the Department of Computer Science, City University of Hong Kong, Hong Kong 999077, China (e-mail: chenning@ieee.org).

Tie Qiu is with the School of Computer Science and Engineering, Northeastern University, Shenyang 110819, China (e-mail: qiutie@ieee.org).

topologies, DIFF-TRO incorporates a constraint-aware decoding strategy that explicitly enforces geometric feasibility and connectivity requirements, yielding topologies with enhanced structural robustness. These components are further integrated into a closed-loop refinement framework that iteratively generates candidate topologies, evaluates their robustness, and feeds high-robustness samples back into the training process, thereby progressively refining the learned diffusion prior under data-scarce settings.

The main contributions of this paper are summarized as follows:

- We propose DIFF-TRO, a self-supervised diffusion framework for IoT topology robustness optimization that learns robustness-oriented structural priors from unlabeled topologies to mitigate path dependence.
- We introduce a constraint-aware decoding strategy that systematically integrates IoT-specific feasibility and connectivity constraints into the topology reconstruction process, thereby producing feasible and high-robustness network topologies from denoised latent representations.
- We design a closed-loop iterative mechanism that alternates between generation, robustness evaluation, and dataset augmentation, progressively refining the learned diffusion prior through robustness-driven feedback.

The remainder of this paper is organized as follows. Section II presents the preliminaries and reviews related work. Section III provides an overview of the DIFF-TRO framework and its mathematical formulation. Section IV details the model design, including topology embedding, diffusion, decoding, and closed-loop training. Section V describes the algorithmic implementation. Section VI presents the performance evaluation, and Section VII concludes the paper with future research directions.

II. Preliminaries and Related Work

This section introduces the quantitative metric used to evaluate the robustness of IoT topologies and reviews representative approaches to topology robustness optimization, including search-based heuristic algorithms and data-driven neural network prediction models. These discussions establish the theoretical foundation for the proposed diffusion-based framework presented in Section III.

A. Topology Robustness for IoT

To quantify the resilience of an IoT topology under node failures or targeted attacks, we adopt the robustness metric proposed by Schneider *et al.* [20]. Given a network with N nodes, nodes are removed sequentially according to a High-Degree Attack (HDA). After the i-th removal, the size of the largest connected component is denoted by $S\left(\frac{i}{N}, x_i\right)$. The robustness metric R is defined as

$$R = \frac{1}{N} \sum_{i=1}^{N} S\left(\frac{i}{N}, x_i\right). \tag{1}$$

which corresponds to the normalized area under the LCC–removal curve. A higher value of R indicates slower connectivity degradation and thus stronger structural robustness. The metric satisfies $R \in (0, 0.5]$, with $R = 0.5$ achieved by a fully connected network, and serves as a standard robustness measure in IoT topology studies [11], [14], [15].

B. Search-Based Heuristic Algorithms

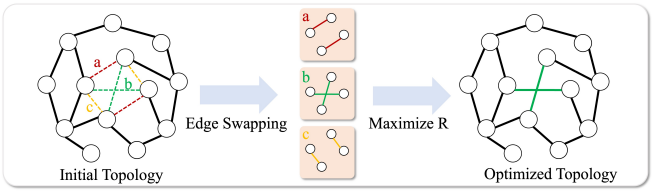
Early studies on topology robustness optimization primarily rely on search-based heuristic algorithms, which iteratively modify the edge structure of a network to enhance robustness while preserving connectivity constraints. Typical examples include the Hill-Climbing Algorithm (HCA) [12] and the evolutionary optimization framework ROCKS [13]. HCA greedily performs edge swaps that yield the largest incremental gain in robustness R, whereas ROCKS introduces crossover and mutation operations to maintain population diversity and prevent premature convergence.

While such approaches are conceptually straightforward and interpretable, they encounter significant scalability limitations when applied to large-scale IoT networks. Each iteration requires re-evaluating the robustness metric across numerous edge configurations, resulting in cubic or higher computational complexity with respect to the number of nodes. Moreover, because these algorithms rely on local edge adjustments, their optimization process is inherently constrained by path dependence [17]: early edge decisions restrict the search space and often lead to convergence toward suboptimal local structures. Reinforcement learning (RL)-based variants [14], [15], [16] have been proposed to improve search efficiency by learning edge-rewiring policies; however, they still require frequent interaction with the environment and repeated robustness evaluations, leading to high time consumption. These challenges make heuristic and RL-based methods inefficient for optimizing large, dense IoT topologies.

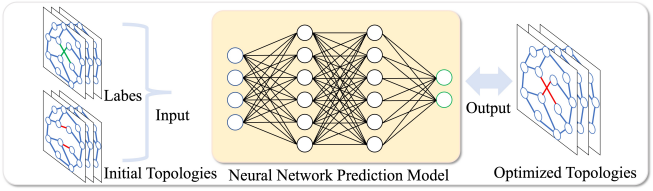
C. Data-Driven Neural Network Prediction Algorithms

Data-driven approaches aim to learn global structural patterns directly from data, providing an alternative to iterative heuristic search. Early studies such as [18] trained supervised neural models to predict topology-wide edge distributions from labeled pairs, enabling parallel generation and offering a conceptual means to mitigate path dependence by bypassing locally accumulated rewiring decisions. More broadly, denoising diffusion probabilistic models (DDPMs) [21], [22], [23], [24], [25] and their graph-based extensions [26], [27], [28], [29] further expand this paradigm by learning distributions over graph structures through iterative corruption-denoising in continuous latent spaces, thereby providing a principled mechanism for exploring combinatorial topology design at the distributional level.

Despite these conceptual advantages, existing data-driven methods remain insufficient for practical IoT topology robustness optimization. First, most models rely on large labeled collections of robust topologies, which are rarely obtainable in real IoT deployments due to privacy constraints, device heterogeneity, and dynamic operational conditions, leading to data scarcity and limited generalization [19]. Second, current supervised predictors and graph-diffusion frameworks are designed for general-purpose tasks and lack decoding mechanisms that



(a) Search-based heuristic edge-swapping process.



(b) Data-driven neural network prediction process.

Fig. 1: Comparison between heuristic and data-driven topology optimization approaches. (a) Heuristic methods iteratively rewire edges to enhance robustness under connectivity constraints. (b) Neural models directly learn mappings from initial to optimized topologies using labeled datasets, enabling global pattern learning but requiring large-scale data.

enforce IoT-specific constraints, such as spatial feasibility, communication range limits, degree balance, and component connectivity [26], [27], [28], [29]. Without these domain-aware constraints, generated graphs often become infeasible or structurally suboptimal for real-world deployment.

In summary, while data-driven models provide a theoretical basis for mitigating path dependence by learning global or distributional structural priors, they do not intrinsically resolve the challenges of label scarcity and constraint-aware topology decoding. These limitations underscore the need for a self-supervised, constraint-integrated generative framework capable of learning robustness-related priors directly from unlabeled IoT networks—an objective addressed in the proposed DIFF-TRO framework.

III. Overview of DIFF-TRO

TABLE I: Notation and Parameter Definitions

Symbol	Meaning	Comment
S	500 m	Deployment area side length
N	100–500	Number of nodes
D	200 m	Communication range
M	2–6	Edge density (avg. degree)
$\mathcal{E}_{\text{feas}}$	–	Feasible edges: $\text{dis}(v_i, v_j) \le D$
A_{ij}	$\{0,1\}$	Binary adjacency
$R(G)$	$(0, 0.5]$	Robustness metric (1)

To overcome the limitations of search-based and supervised learning approaches in IoT topology optimization, DIFF-TRO is designed as a diffusion-driven, self-supervised framework that reformulates topology generation as a denoising process in a continuous latent space. By combining latent diffusion, domain-aware decoding, and closed-loop refinement, DIFF-TRO learns robustness-related structural priors directly from unlabeled topologies and progressively improves them through closed-loop robustness feedback.

A. Problem Statement

Given an IoT network represented by $G = (V, E)$, where each node $v_i \in V$ is associated with a geographic position $\text{pos}(v_i) \in \mathbb{R}^2$. Edges are permitted only when the wireless communication constraint $\text{dis}(v_i, v_j) \le D$ is satisfied, yielding the feasible edge set:

$$\mathcal{E}_{\text{feas}} = \big\{(i,j) \mid \text{dis}(v_i, v_j) \le D\big\}. \tag{2}$$

Thus, the robustness optimization problem is formulated as:

$$\begin{aligned} \underset{E^* \subseteq \mathcal{E}_{\text{feas}}}{\text{maximize}} \quad & R(G^*) = \frac{1}{N}\sum_{i=1}^{N} S\left(\frac{i}{N}, x_i\right), \\ \text{s.t.} \quad & \begin{cases} |E^*| = |E_0|, \\ A_{ij} = A_{ji} \in \{0,1\}, \\ \text{pos}(v_i) \in S, \\ \text{dis}(v_i, v_j) \le D, \end{cases} \end{aligned} \tag{3}$$

where $G^* = (V, E^*)$ is the optimized topology and $R(G)$ is the robustness metric defined in (1). This formulation requires preserving spatial feasibility, fixed node positions, communication constraints, and symmetric links while maximizing network robustness under targeted attacks.

Unlike existing search-based methods that attempt to solve (3) through discrete edge rewiring and are therefore fundamentally constrained by path dependence, DIFF-TRO learns a generative diffusion prior in a continuous latent space and transforms its denoised latent representations into feasible and robust topologies. This continuous formulation enables global structure exploration, mitigates path dependence, and allows domain-aware constraints to be incorporated through a dedicated decoding mechanism and a closed-loop iterative refinement process.

B. Framework of DIFF-TRO

Fig. 2 illustrates the high-level workflow of DIFF-TRO. Rather than modifying individual edges, DIFF-TRO learns robustness-centric structural patterns through four interconnected stages:

1) Initial Topology: Generate unlabeled, geometrically feasible IoT graphs with fixed node density and wireless communication constraints.
2) Latent Embedding: Encode node coordinates, edge relations, and diffusion timestep into a spatio-temporal latent space, making discrete topologies differentiable.
3) Diffusion-Based Denoising: Inject and iteratively remove Gaussian noise via an anisotropic GNN, recovering latent adjacency structures aligned with robustness-enhancing priors.
4) Decoding and Closed-Loop Reinforcement: Apply degree-balancing and component-bridging constraints to form valid IoT topologies.

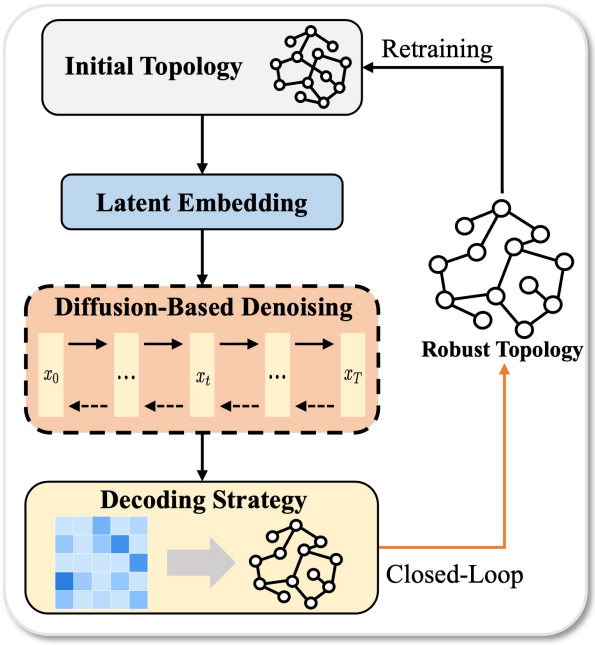


Fig. 2: Overview of the DIFF-TRO framework, showing embedding, diffusion, decoding, and closed-loop reinforcement for robustness-oriented topology generation.

Unlike standard generative models, DIFF-TRO is not a one-shot sampler. Instead, it forms a closed-loop, robustness-driven learning cycle: sampling → decoding → robustness evaluation → dataset expansion → retraining. This enables DIFF-TRO to progressively converge toward robustness-optimal structural priors without labeled supervision. The workflow in Fig. 2 also clarifies the division of responsibility among the modules. The embedding stage provides a differentiable representation of spatially constrained IoT graphs; the diffusion stage learns a distributional prior over robust structures; the decoding stage maps continuous confidence scores back to feasible edges; and the closed-loop stage uses robustness feedback to enrich the training set. This separation allows each component to be analyzed independently while still forming a unified optimization pipeline.

To reveal internal architectural details, Fig. 3 further decomposes DIFF-TRO into spatio-temporal topology embedding, anisotropic diffusion denoising, and constraint-aware decoding modules.

IV. DIFF-TRO

This section presents the core methodology of the proposed DIFF-TRO framework. DIFF-TRO operates through four sequential stages: (1) initial topology generation, (2) topology embedding, (3) diffusion-based denoising, and (4) constraint-aware decoding strategy, integrated within a closed-loop self-supervised training strategy. Together, these components enable the learning of robustness-oriented structural priors directly from unlabeled IoT topologies while ensuring geometric feasibility.

A. Initial Topology Generation

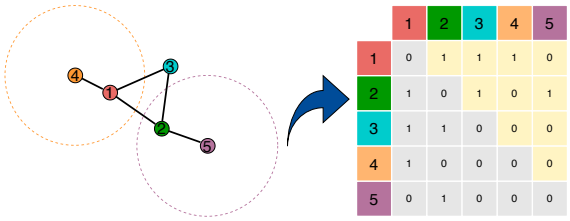


Fig. 4: Examples of initial topologies generated under geometric and communication constraints. Nodes are placed in a bounded spatial region and edges are formed according to distance feasibility.

DIFF-TRO begins with constructing a set of initial topologies $\{G_0^k = (V^k, E_0^k)\}_{k=1}^K$, obtained either from real IoT deployments or synthesized via random geometric graphs under wireless communication constraints. Each node v_i is assigned a fixed spatial position $x_i \in S \subset \mathbb{R}^2$, and an edge (i, j) is feasible only if the Euclidean distance satisfies $\mathrm{dis}(x_i, x_j) \leq D$. The feasible edge set is therefore defined as (2). To ensure comparability across optimization runs, the average degree $M = |E_0|/|V|$ is kept constant.

These initial graphs contain diverse spatial layouts, degree patterns, and connectivity structures, providing heterogeneous seed samples for diffusion-based learning. Instead of performing rule-driven edge rewiring, DIFF-TRO treats these graphs as samples drawn from an underlying latent distribution of potentially robust IoT topologies. The aim of diffusion modeling is to learn this structural distribution and refine it through denoising. By embedding geometric constraints directly into the feasible edge space, DIFF-TRO maintains physical realizability while enabling continuous latent-space optimization.

B. Topology Embedding

To enable differentiable processing of discrete graph structures, each topology is mapped into a continuous latent space through joint node, edge, and timestep embeddings. This embedding stage preserves spatial geometry, structural connectivity, and temporal diffusion semantics, forming the foundation of the generative denoising process.

a) Node Embedding: Each node position $x_i \in \mathbb{R}^2$ is encoded using a sinusoidal positional encoder:

$$\mathbf{h}_i^{(0)} = [\sin(\omega x_i), \cos(\omega x_i)]\mathbf{W}_n, \tag{4}$$

where ω is a learnable frequency vector and $\mathbf{W}_n \in \mathbb{R}^{2d \times d_h}$ aligns positional features with the hidden dimension. This encoding captures fine-grained geometric structure while providing translation-invariant positional cues.

b) Edge Embedding: For each feasible pair (i, j), the adjacency value $A_{ij} \in \{0, 1\}$ (or a normalized distance weight) is transformed into a continuous relation embedding:

$$\mathbf{e}_{ij}^{(0)} = [\sin(\omega A_{ij}), \cos(\omega A_{ij})]\mathbf{W}_e, \tag{5}$$

where $\mathbf{W}_e$ projects pairwise connectivity information into the latent dimension. This representation captures structural relations essential for downstream message passing.

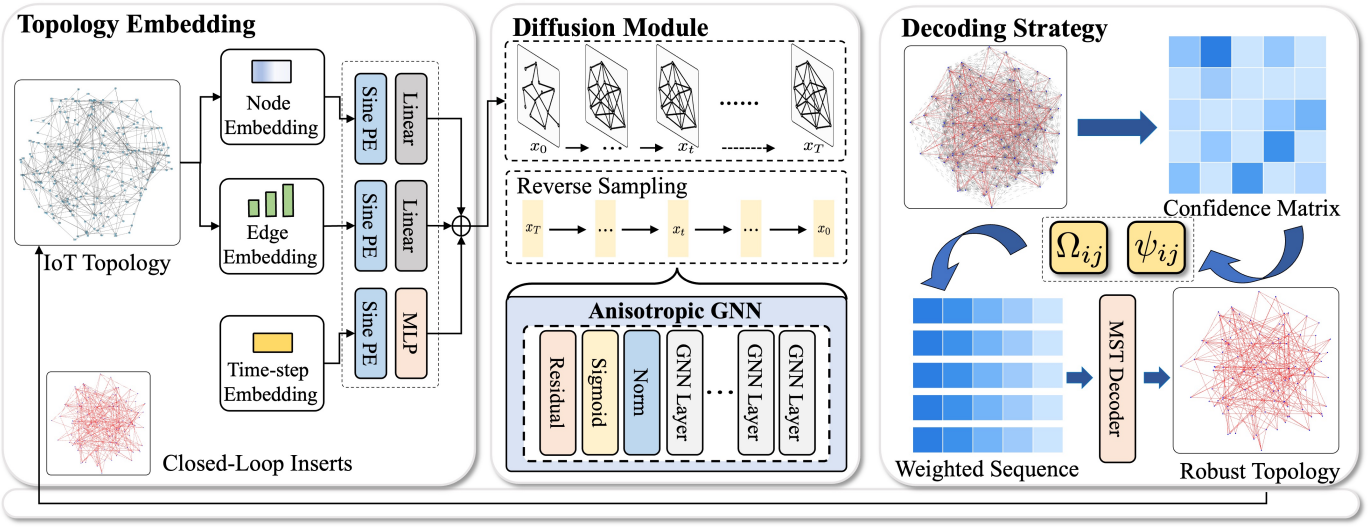


Fig. 3: Internal architectural design of DIFF-TRO, illustrating joint node–edge–timestep embedding, anisotropic diffusion denoising, and constraint-aware decoding for reconstructing feasible and robustness-optimized topologies.

c) Timestep Embedding: The diffusion process operates over timesteps $t \in \{1, \dots, T\}$, each encoded as

$$\mathbf{z}_t = [\sin(\omega t),\, \cos(\omega t)]\mathbf{W}_t, \tag{6}$$

with $\mathbf{W}_t$ serving as a lightweight projection. The timestep embedding is added to both node and edge features, enabling the model to distinguish noise levels and track progress along the denoising trajectory.

Collectively, these embeddings form the unified latent representation $(\mathbf{H}^{(0)}, \mathbf{E}^{(0)})$, which encodes topology geometry, connectivity, and diffusion-stage context for subsequent generative modeling.

C. Diffusion Module

The diffusion module models the latent structural distribution of robust IoT topologies through a forward–reverse generative process [21], [22], [23]. Here, *self-supervised* refers to denoising-based reconstruction: the model learns to recover clean topology embeddings from noised counterparts without external robustness labels. In the forward process, Gaussian noise is gradually injected into the topology embeddings, producing a noisy latent representation $\hat{x}_T$ that approximates an isotropic Gaussian distribution. Formally,

$$q(\hat{x}_t|\hat{x}_{t-1}) = \mathcal{N}\big(\hat{x}_t; \sqrt{1-\beta_t}\,\hat{x}_{t-1},\, \beta_t \mathbf{I}\big), \tag{7}$$

where $\beta_t \in (0,1)$ denotes the variance schedule. By recursively expanding the transition kernel, the closed-form t-step marginal becomes

$$q(\hat{x}_t|\hat{x}_0) = \mathcal{N}\big(\hat{x}_t; \sqrt{\bar{\alpha}_t}\hat{x}_0,\, (1-\bar{\alpha}_t)\mathbf{I}\big), \tag{8}$$

with $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{\tau=1}^{t} \alpha_\tau$.

The reverse process iteratively reconstructs clean topology embeddings by predicting $\hat{x}_0$ from the noisy sample $\hat{x}_t$ using a parameterized denoiser f_θ:

$$p_\theta(\hat{x}_{t-1}|\hat{x}_t) = q\left(\hat{x}_{t-1} \,\middle|\, \hat{x}_t,\, \hat{x}_0 = \frac{\hat{x}_t - \sqrt{1-\bar{\alpha}_t}\, f_\theta(\hat{x}_t, t)}{\sqrt{\bar{\alpha}_t}}\right). \tag{9}$$

Through this denoising process, DIFF-TRO refines latent structures toward higher robustness. Algorithm 2 uses deterministic $\hat{x}_0$-prediction (DDIM with $\eta = 0$), so the stochastic $\sqrt{\beta_t}\mathbf{z}$ term is omitted.

a) AGNN Backbone: To capture anisotropic structural cues at both node and edge levels, DIFF-TRO employs an anisotropic graph neural network (AGNN) [30] as the diffusion denoiser. Let $\mathbf{h}_i^\ell$ and $\mathbf{e}_{ij}^\ell$ denote node and edge embeddings at layer ℓ. The AGNN update is given by:

$$\begin{aligned}
\hat{\mathbf{e}}_{ij}^{\ell+1} &= \mathbf{A}^\ell \mathbf{h}_i^\ell + \mathbf{B}^\ell \mathbf{h}_j^\ell + \mathbf{C}^\ell \mathbf{e}_{ij}^\ell, \\
\mathbf{e}_{ij}^{\ell+1} &= \mathbf{e}_{ij}^\ell + \mathrm{ReLU}\big(\mathcal{BN}(\hat{\mathbf{e}}_{ij}^{\ell+1})\big) + \mathrm{MLP}(\mathbf{z}_t), \\
\mathbf{h}_i^{\ell+1} &= \mathbf{h}_i^\ell + \mathrm{ReLU}\Big(\mathcal{BN}\big(\mathbf{U}^\ell \mathbf{h}_i^\ell + \sum_{j \in \mathcal{N}_i} \sigma(\hat{\mathbf{e}}_{ij}^{\ell+1}) \mathbf{V}^\ell \mathbf{h}_j^\ell\big)\Big).
\end{aligned} \tag{10}$$

where $\sigma(\cdot)$ denotes the sigmoid function, $\mathcal{BN}(\cdot)$ is batch normalization, $\mathrm{ReLU}(\cdot)$ is the rectified linear activation, and $\mathrm{MLP}(\mathbf{z}_t)$ injects timestep information.

This architecture integrates geometric locality, structural connectivity, and temporal diffusion context, enabling the effective refinement of latent topological structures across denoising steps.

D. Constraint-aware Decoding Strategy

The denoising module outputs a dense confidence matrix $\tilde{A}$, where $\tilde{A}_{ij} \in [0,1]$ approximates the probability of forming an edge between nodes i and j. To convert this latent representation into a feasible IoT topology, DIFF-TRO applies domain-aware decoding operators that incorporate degree balance, component bridging, and geometric feasibility. Hereafter, A_{ij} denotes binary adjacency, $\tilde{A}_{ij}$ denotes latent confidence, and A'_{ij} denotes normalized decoding scores. We first apply a smooth normalization:

$$A'_{ij} = \frac{\tanh(\tilde{A}_{ij}) + 1}{2}, \tag{11}$$

which stabilizes the confidence values and suppresses overly aggressive activations.

a) Degree-Balancing Operator: To discourage excessive concentration of edges on high-degree nodes, a degree-aware weight is introduced:

$$\Omega_{ij} = \frac{\exp\left[-(\Delta d_{ij}/d_{\max})^2\right]}{\sqrt{d_i d_j}}, \qquad \Delta d_{ij} = |d_i - d_j|. \quad (12)$$

This operator encourages a more uniform degree distribution, a structural trait strongly correlated with robustness [31].

b) Bridging Operator: To enhance global connectivity, edges connecting different components receive higher priority:

$$\psi_{ij} = \begin{cases} \sqrt{|\mathcal{C}_u| \cdot |\mathcal{C}_v|}, & u \neq v, \\ 1, & u = v. \end{cases} \quad (13)$$

where u and v denote the connected components containing nodes i and j, respectively. This operator promotes cross-component linkage and mitigates fragmentation, both of which are essential for robustness under node removals [32].

c) Scoring and Topology Reconstruction: The final edge score is computed as:

$$\mathrm{Score}(i,j) = A'_{ij} \cdot \Omega_{ij} \cdot \psi_{ij}. \quad (14)$$

DIFF-TRO constructs a minimum spanning tree (MST) to guarantee baseline connectivity, followed by the sequential addition of the highest-scoring feasible edges until $|E^*| = |E_0|$. The resulting topology $G^* = (V, E^*)$ satisfies geometric constraints, degree conditions, and exhibits enhanced robustness.

E. Closed-Loop Iterative Training

To progressively strengthen the learned diffusion prior, DIFF-TRO employs a closed-loop self-supervised training mechanism. At round r, all n_r generated topologies $\{G_i^{*(r)}\}_{i=1}^{n_r}$ are evaluated by (1). The threshold τ_r is set to the $(1-p)$-quantile of their robustness metric values:

$$\tau_r \;=\; \mathrm{Quantile}_{1-p}\Big(\{R(G_i^{*(r)})\}_{i=1}^{n_r}\Big), \quad (15)$$

The dataset is then expanded by admitting candidates whose robustness is no smaller than τ_r:

$$\mathcal{D}_{r+1} = \mathcal{D}_r \cup \{\, G_i^{*(r)} \mid R(G_i^{*(r)}) \geq \tau_r \,\}. \quad (16)$$

Throughout the paper, $p = 10$ is used unless stated otherwise. The rule is order-independent: it depends only on the set of robustness values, not on the order in which candidates are evaluated.

Through this iterative feedback, DIFF-TRO continuously refines its generative capability, enhancing its ability to model robustness-oriented structural priors without requiring labeled data. The closed-loop mechanism thus enables robustness-driven iterative improvement and scalability across diverse IoT network settings.

V. Algorithm Design

This section gives pseudocode for the four DIFF-TRO modules introduced in Section IV: topology embedding, diffusion-based denoising, constraint-aware decoding, and closed-loop iterative training. Each module is presented with its key variables and a short explanation of its algorithmic role.

A. Topology Embedding

Algorithm 1 describes how DIFF-TRO transforms discrete IoT topologies into continuous latent embeddings, enabling differentiable learning. The variables used in the algorithm are summarized as follows:

- X: Spatial coordinates of all nodes in the deployment region.
- A: Binary adjacency matrix describing observed connectivity.
- ω: Frequency coefficient used in sinusoidal positional encoders.
- $\mathbf{h}_i^{(0)}, \mathbf{e}_{ij}^{(0)}$: Initial node and edge embeddings constructed from spatial and structural features.
- $\mathbf{z}_t$: Timestep embedding injected into all representations to encode diffusion-stage information.
- $\mathbf{H}^{(0)}, \mathbf{E}^{(0)}$: Final initial embeddings provided to the diffusion denoiser.

Algorithm 1 Topology Embedding Generation

Input: Node coordinates $X = \{x_i\}$, adjacency A, diffusion step t
Output: Embedded node & edge features $\mathbf{H}^{(0)}, \mathbf{E}^{(0)}$

// Node Embedding:
for each $v_i \in V$ **do**
  $\mathbf{h}_i^{(0)} \leftarrow [\sin(\omega x_i), \cos(\omega x_i)]W_n$
end for
// Edge Embedding:
for each feasible pair (i,j) **do**
  $\mathbf{e}_{ij}^{(0)} \leftarrow [\sin(\omega A_{ij}), \cos(\omega A_{ij})]W_e$
end for
// Timestep Embedding:
$\mathbf{z}_t \leftarrow [\sin(\omega t), \cos(\omega t)]W_t$
for each i and (i,j) **do**
  $\mathbf{h}_i^{(0)} \leftarrow \mathbf{h}_i^{(0)} + \mathbf{z}_t, \quad \mathbf{e}_{ij}^{(0)} \leftarrow \mathbf{e}_{ij}^{(0)} + \mathbf{z}_t$
end for
return $\mathbf{H}^{(0)}, \mathbf{E}^{(0)}$

Algorithm 1 constructs the initial latent representation by encoding node locations, adjacency values, and diffusion timestep information into continuous embeddings. The sinusoidal encoders preserve spatial structure, and the shared timestep embedding synchronizes node and edge representations across denoising iterations. These embeddings form the unified input to DIFF-TRO's diffusion-based denoising module and enable differentiable learning over originally discrete graph structures.

B. Diffusion-Based Denoising

Algorithm 2 defines the forward and reverse diffusion procedures used to denoise topology embeddings. The variables involved in the diffusion procedure are listed below:

- x_t: Latent adjacency representation at timestep t.
- β_t: Variance value for the forward noise schedule.
- $\alpha_t = 1 - \beta_t$: Noise-retention coefficient for timestep t.
- $\hat{x}_0$: Clean latent adjacency predicted by the AGNN.
- f_θ: Parameterized AGNN backbone for reverse denoising.

- $\tilde{A}$: Final denoised adjacency-confidence matrix produced at $t = 0$.

For brevity, Algorithm 2 writes the latent adjacency as x_t; the theoretical notation $\hat{x}_t$ only emphasizes stochasticity.

Algorithm 2 Diffusion Denoising with AGNN Backbone

Input: Embedded features $(\mathbf{H}^{(0)}, \mathbf{E}^{(0)})$, noise schedule $\{\beta_t\}_{t=1}^{T}$, AGNN model f_θ
Output: Denoised latent adjacency $\tilde{A}$
Initialize $x_0 \leftarrow \mathbf{E}^{(0)}$
Forward Diffusion:
for $t = 1$ to T **do**
  $x_t \sim \mathcal{N}(\sqrt{1-\beta_t}\, x_{t-1},\, \beta_t I)$
end for
Reverse Denoising:
for $t = T$ down to 1 **do**
  $\hat{x}_0 \leftarrow f_\theta(x_t, t)$
  $x_{t-1} \leftarrow \frac{1}{\sqrt{\alpha_t}}\,(x_t - (1-\alpha_t)\hat{x}_0)$ ▷ deterministic $\hat{x}_0$-prediction update; DDIM $\eta{=}0$, no noise term
end for
return $\tilde{A} = \sigma(\mathrm{Conv2D}(x_0))$

Algorithm 2 summarizes the denoising process: forward noise smooths the adjacency distribution, and AGNN-guided reverse steps recover robustness-aware latent structure from unlabeled topologies.

C. Constraint-aware Decoding

Algorithm 3 converts $\tilde{A}$ into a feasible IoT topology through constrained edge ranking. The decoding variables are defined as follows:

- $\tilde{A}$: Denoised latent adjacency matrix generated by the diffusion module.
- A'_{ij}: Normalized confidence score obtained after activation smoothing.
- Ω_{ij}: Degree-balancing weight encouraging uniform degree distribution.
- ψ_{ij}: Component-bridging weight promoting inter-component connectivity.
- $\mathrm{Score}(i,j)$: Composite edge score integrating structural and geometric factors.
- E_{MST}: Minimum spanning tree ensuring baseline global connectivity.
- E^*: Final reconstructed edge set satisfying feasibility and cardinality constraints.

Algorithm 3 enforces IoT feasibility through MST connectivity and then ranks candidate edges with degree-balancing and component-bridging priors.

D. Closed-Loop Iterative Training

Algorithm 4 updates $\mathcal{D}_r$ by scoring decoded candidates and retaining those above threshold τ_r. The variables used in the closed-loop stage are defined as follows:

- $\hat{A}^{(k)}$: Candidate adjacency matrix produced by diffusion sampling.

Algorithm 3 Edge Ranking and Robust Topology Decoding

Input: $\tilde{A}$, initial degree d_i, components $\{\mathcal{C}_k\}$, target $|E_0|$
Output: Optimized topology $G^* = (V, E^*)$
$A'_{ij} \leftarrow (\tanh(\tilde{A}_{ij}) + 1)/2$
$\Omega_{ij} \leftarrow DegreeBalance(d_i, d_j)$ ▷ Eq. (12)
$\psi_{ij} \leftarrow ComponentBridge(i, j, \{\mathcal{C}_k\})$ ▷ Eq. (13)
$\mathrm{Score}(i,j) = A'_{ij} \cdot \Omega_{ij} \cdot \psi_{ij}$
Build MST using $-A'_{ij}$; initialize $E^* = E_{\mathrm{MST}}$
for edges (i,j) in descending Score order **do**
  if $|E^*| < |E_0|$ **and** $(i,j) \notin E^*$ **then**
    $E^* \leftarrow E^* \cup \{(i,j)\}$
  end if
end for
return $G^* = (V, E^*)$

- $G^{(k)}$: Decoded topology candidate.
- R_k: Robustness value assigned to $G^{(k)}$.
- τ_r: Adaptive robustness threshold at round r.
- $\mathcal{H}$: Retained high-robust topology set.
- $\mathcal{D}_r$: Training dataset before the round-r update.

Algorithm 4 Closed-Loop Robustness Evaluation and Dataset Expansion

Input: Candidate set $\{\hat{A}^{(k)}\}_{k=1}^{n_r}$, node coordinates X, current dataset $\mathcal{D}_r$, retention ratio p
Output: Updated dataset $\mathcal{D}_{r+1}$
$\mathcal{H} \leftarrow \emptyset$
for $k = 1$ **to** n_r **do**
  $G^{(k)} \leftarrow \mathrm{DecodeAdjMat}(\hat{A}^{(k)}, X)$ ▷ Algorithm 3
  $R_k \leftarrow \mathrm{CalculateRobustness}(G^{(k)})$ ▷ Eq. (1)
end for
$\tau_r \leftarrow \mathrm{Quantile}_{1-p}(\{R_k\}_{k=1}^{n_r})$
for $k = 1$ **to** n_r **do**
  if $R_k \geq \tau_r$ **then**
    Append $G^{(k)}$ to $\mathcal{H}$
  end if
end for
$\mathcal{D}_{r+1} \leftarrow \mathcal{D}_r \cup \mathcal{H}$
return $\mathcal{D}_{r+1}$

Algorithm 4 uses Eq. (1) to retain robust samples, so $\mathcal{D}_r$ accumulates robustness priors without labels.

VI. Performance Evaluation

This section evaluates DIFF-TRO under a range of IoT network configurations. It reports the parameter settings, convergence behavior, baseline comparison, graph attributes, topology visualization, and ablation results. All results are reported as mean ± standard deviation over 10 independent runs.

A. Parameter Settings

All experiments were conducted using PyTorch 2.2 on an Ubuntu 22.04 server with a 48-core Intel Xeon Silver 4310

CPU (2.10 GHz) and two NVIDIA A40 GPUs. We compared DIFF-TRO with five baselines: HCA [12], ROCKS [13], BTW [33], UNITY [34] and ROMEM [15]. UNITY appears in the robustness figures but is omitted from the runtime tables because those tables compare iterative search/refinement methods. The BP-NN [18] model was omitted due to inferior robustness performance compared to its own training set.

Each setting was tested under 10 randomized initializations to ensure statistical reliability. All nodes were randomly distributed in a $500\,\mathrm{m}\times500\,\mathrm{m}$ area with maximum communication distance $D = 200\,\mathrm{m}$.

TABLE II: Simulation Parameters and Hyperparameters.

Parameter	Value / Range	Description
S	$500\,\mathrm{m}$	Deployment area side length
N	100–500	Number of nodes
M	2–6	Edge density (avg. degree)
D	$200\,\mathrm{m}$	Communication range
L	12	GNN layers
H	256	Hidden dimension
Agg	sum	Aggregation function
lr	2×10^{-4}	Learning rate
T	1000	Diffusion steps
Batch	32	Batch size

B. Training Convergence

Fig. 5 shows that DIFF-TRO converges within roughly 100 epochs across all nine (N, M) settings. The validation curves track the training curves without persistent divergence, showing that the diffusion denoiser learns a stable mapping under both scale and density variation.

C. Diffusion and Denoising Behavior Analysis

We analyze the reverse diffusion process from two complementary views: the evolution of the adjacency-confidence matrix $\tilde{\mathbf{A}}_t$ and its normalized Frobenius discrepancy from the seed adjacency $\mathbf{A}$:

$$d_t \;=\; \tfrac{1}{N}\big\|\tilde{\mathbf{A}}_t - \mathbf{A}\big\|_F. \tag{17}$$

Figs. 6 and 7 show that the confidence matrix evolves from a near-uniform distribution to a clear edge skeleton, while d_t decreases through the main reverse steps with only a mild late rebound. This indicates that DIFF-TRO learns a non-trivial structural prior rather than merely smoothing noise, and it motivates decoding from an intermediate timestep $t^\star$ instead of $t=0$.

D. Embedding Ablation and Edge-Level Discriminability

We ablate the three spatio-temporal embedding components under the same AGNN and training protocol: "*w/o Pos*" uses learnable random node embeddings, "*w/o Edge*" zeros edge-relation encoding, and "*w/o Time*" removes timestep injection in (10). The "*w/o Edge*" setting also serves as a uniform-prior control because $\tilde{\mathbf{A}}$ becomes nearly uninformative while the decoder is unchanged.

Fig. 8 and Table III show that edge-relation encoding is indispensable: removing it drives d^* toward the random-matrix baseline (≈ 0.707) and AUC to chance (≈ 0.50). Removing node-position encoding also reduces edge-level discriminability, while timestep encoding is more important for denser topologies. These results attribute the R gain in Section VI-E to the diffusion-learned prior rather than the decoder alone.

TABLE III: Embedding ablation: matrix-level discrepancy d^* (↓) and edge-level AUC (↑). Mean±std over 10 runs.

Setting	Variant	$d^*\downarrow$	AUC↑
$N=100, M=2$	Full	$\mathbf{0.312\pm0.004}$	0.562 ± 0.002
	w/o Pos	0.507 ± 0.000	0.496 ± 0.008
	w/o Edge	0.694 ± 0.001	0.504 ± 0.000
	w/o Time	0.511 ± 0.001	$\mathbf{0.615\pm0.010}$
$N=100, M=6$	Full	0.511 ± 0.003	$\mathbf{0.752\pm0.005}$
	w/o Pos	$\mathbf{0.444\pm0.001}$	0.503 ± 0.005
	w/o Edge	0.700 ± 0.000	0.498 ± 0.000
	w/o Time	0.550 ± 0.001	0.703 ± 0.001
$N=500, M=2$	Full	0.359 ± 0.005	$\mathbf{0.514\pm0.005}$
	w/o Pos	$\mathbf{0.348\pm0.002}$	0.498 ± 0.003
	w/o Edge	0.708 ± 0.000	0.491 ± 0.003
	w/o Time	0.594 ± 0.076	0.519 ± 0.013

E. Optimization under Varying Network Densities and Sizes

We first evaluated networks with a fixed node count $N = 100$ under different edge densities $M \in \{2,3,4,5,6\}$, and then examined networks with a fixed density $M = 2$ and node counts $N \in \{100, 200, 300, 400, 500\}$. Denser networks tend to preserve larger connected components after node removals, thus achieving higher robustness. Conversely, larger networks show lower robustness due to increased vulnerability to high-degree attacks.

Fig. 9 illustrates the robustness metric R as a function of optimization epochs for both experiments. In both settings, DIFF-TRO rapidly increases R within five epochs and stabilizes thereafter, showing fast convergence and stable performance across diverse IoT topologies.

We next compare DIFF-TRO with ROMEM, BTW, ROCKS, and HCA under the same density and size settings.

a) Varying Edge Density: Fig. 10 shows robustness results for $M \in \{2,3,4,5,6\}$ with $N = 100$. DIFF-TRO consistently outperforms all baselines, while BTW degrades when $M > 3$ due to its local growth-based heuristic.

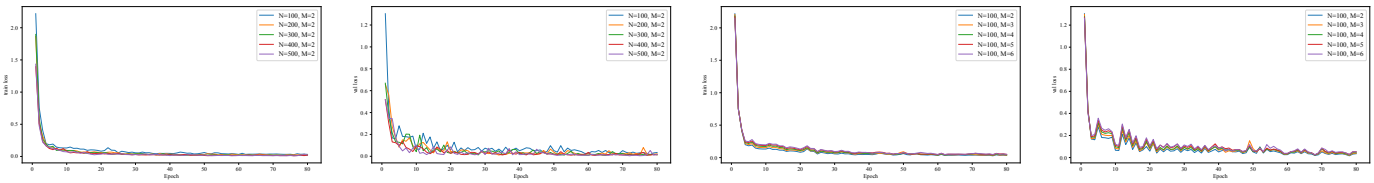
Table IV(a) lists the corresponding runtime. DIFF-TRO runtime includes one diffusion sampling pass and one decoding round per instance; the one-shot offline training cost is amortized and excluded, consistent with the baselines.

b) Varying Network Size: Fig. 11 compares robustness for $N \in \{100, 200, 300, 400, 500\}$ with $M = 2$, where ROMEM and BTW degrade as the graph grows.

Table IV(b) shows that DIFF-TRO scales efficiently with increasing N due to batched graph inference.

F. Stability Compared with ROMEM

Across all density and size configurations, DIFF-TRO achieved an average improvement of $+9.4\% \pm 2.1\%$ in robustness R compared with ROMEM ($p < 0.05$, paired two-



(a) Scale group, training loss (b) Scale group, validation loss (c) Density group, training loss (d) Density group, validation loss

Fig. 5: Loss convergence under scale and density variations (mean±std, log-scale).

TABLE IV: Runtime Comparison Across Varying Edge Densities ($N = 100$) and Network Sizes ($M = 2$). Values denote mean ± std over 10 runs (seconds).

(a) Runtime under varying edge densities ($N = 100$)

Edge Density M	**DIFF-TRO**	**ROMEM**	**BTW**	**ROCKS**	**HCA**
2	724.1 ± 12.4	544.3 ± 10.2	284.5 ± 8.6	968.3 ± 14.7	1690.2 ± 21.3
3	841.2 ± 13.8	631.0 ± 11.5	561.2 ± 9.7	1060.7 ± 15.8	2912.4 ± 27.1
4	915.7 ± 14.6	682.8 ± 12.3	898.3 ± 10.8	1190.6 ± 16.9	4012.8 ± 32.0
5	964.3 ± 15.0	714.5 ± 12.6	1912.1 ± 18.9	1632.4 ± 22.5	4720.5 ± 38.3
6	1341.7 ± 17.4	900.2 ± 14.7	2441.0 ± 21.6	1810.3 ± 24.7	4812.9 ± 40.1

(b) Runtime under varying network sizes ($M = 2$)

Node Count N	**DIFF-TRO**	**ROMEM**	**BTW**	**ROCKS**	**HCA**
100	724.1 ± 12.4	544.3 ± 10.2	284.5 ± 8.6	968.3 ± 14.7	1690.2 ± 21.3
200	6510.3 ± 74.1	4310.2 ± 62.4	3040.8 ± 55.7	4980.5 ± 70.6	32500.1 ± 203.8
300	13100.6 ± 121.5	8610.3 ± 110.9	10900.4 ± 98.6	17390.8 ± 130.1	191800.7 ± 410.3
400	39040.5 ± 200.4	24200.7 ± 185.2	44500.1 ± 222.8	48100.2 ± 245.6	713000.2 ± 650.9
500	50500.6 ± 245.3	30800.1 ± 213.7	165000.3 ± 441.2	101000.4 ± 382.7	1850000.4 ± 1130.3

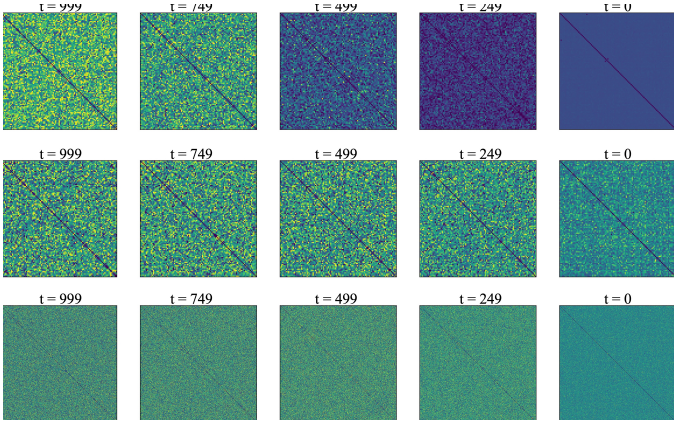


Fig. 6: Confidence-matrix evolution at representative reverse timesteps.

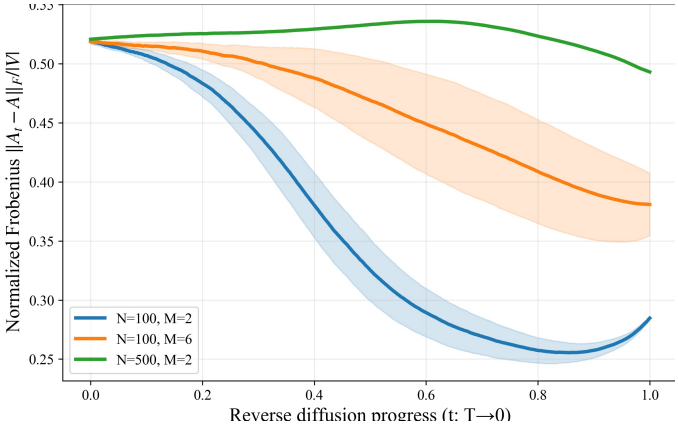


Fig. 7: Normalized Frobenius distance d_t along the reverse trajectory.

tailed t-test). Table V reports the per-configuration paired two-tailed t-test results, where DIFF-TRO and ROMEM are paired on the same random seed and initial topology ($n = 10$ pairs per cell). After Holm–Bonferroni correction across the nine configurations [35], every comparison remains significant at $p < 10^{-4}$; the aggregate gain is therefore not driven by a small subset of settings.

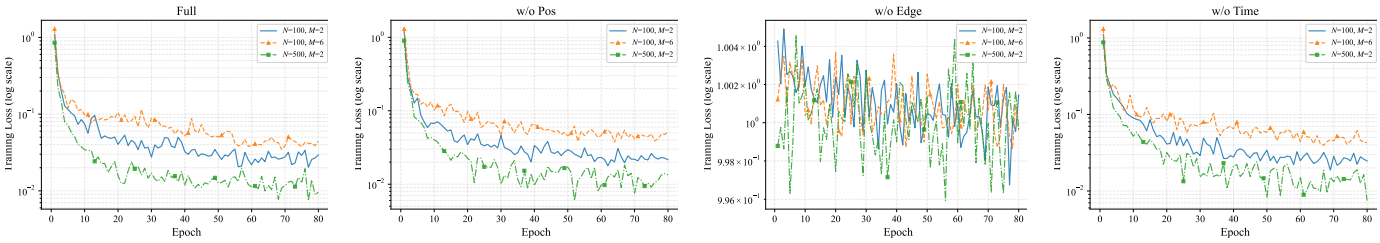
G. Graph Attribute Analysis of Optimized Topologies

We further examine clustering coefficient, average shortest path length, and degree assortativity, which describe local cohesion, global reachability, and degree mixing in sparse IoT graphs. Fig. 12 visualizes their distributions, and Table VI reports the corresponding mean and standard deviation values.

DIFF-TRO consistently increases clustering and assortativity across all network sizes. The path length also rises slightly, indicating a trade-off: decoded topologies introduce more local redundancy and degree-balanced connectivity at the cost of modestly longer routes. These structural shifts support the robustness gains reported above.

H. Ablation Study

To isolate the contribution of each operator in the proposed constraint-aware decoding strategy, we conduct a detailed



(a) Full (b) w/o Pos (c) w/o Edge (d) w/o Time

Fig. 8: Embedding-ablation loss curves (log-scale). The w/o Edge variant fails to learn usable structural signals.

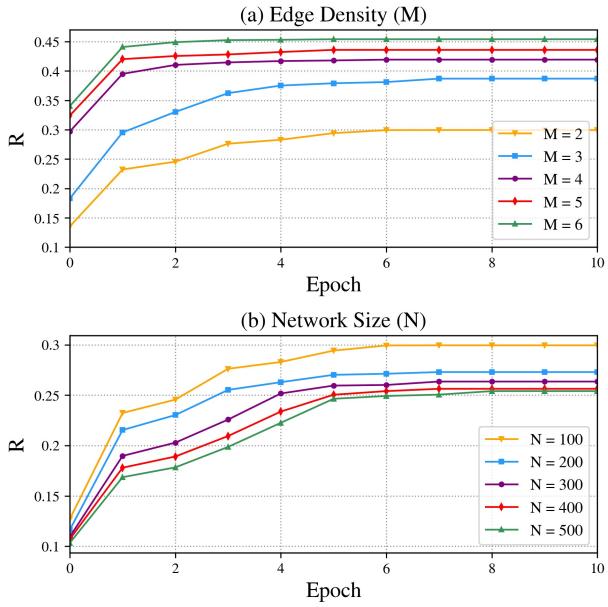


Fig. 9: Optimization performance under varying (a) edge densities and (b) network sizes. Each point represents mean ± std over 10 runs.

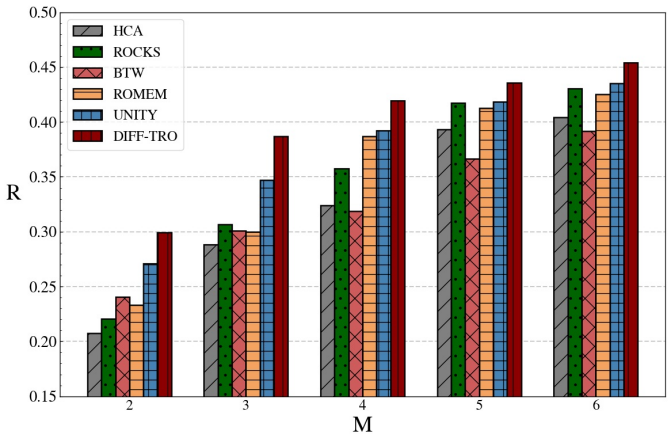


Fig. 10: Robustness comparison under varying edge densities ($N = 100$, mean±std).

ablation analysis focusing on four components: (i) the degree-balancing operator Ω, (ii) the component-bridging operator ψ, (iii) their combination $\Omega + \psi$, and (iv) an additional post-processing step using 2-opt random perturbation. All

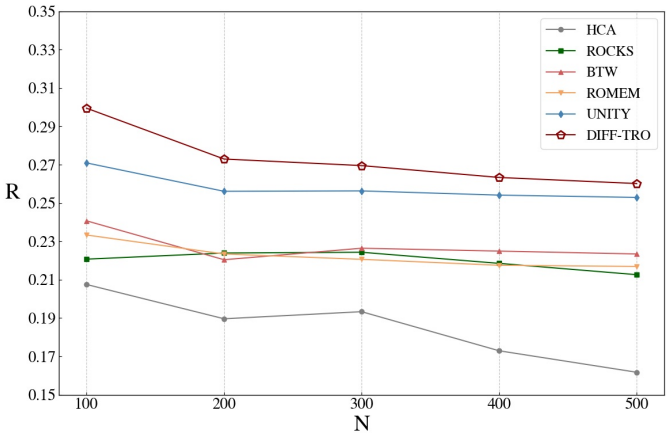


Fig. 11: Robustness comparison under varying network sizes ($M = 2$, mean±std).

TABLE V: Paired t-test results for DIFF-TRO vs. ROMEM ($n = 10$, df = 9).

(N, M)	$\bar{d}$	s_d	t	p_{raw}	p_{Holm}
$(100, 2)$	0.0241	0.0073	10.45	1.4×10^{-6}	$< 10^{-5}$
$(100, 3)$	0.0218	0.0071	9.71	2.1×10^{-6}	$< 10^{-5}$
$(100, 4)$	0.0203	0.0069	9.30	3.0×10^{-6}	$< 10^{-5}$
$(100, 5)$	0.0186	0.0070	8.40	6.6×10^{-6}	$< 10^{-5}$
$(100, 6)$	0.0172	0.0072	7.56	1.4×10^{-5}	$< 10^{-4}$
$(200, 2)$	0.0263	0.0078	10.66	1.2×10^{-6}	$< 10^{-5}$
$(300, 2)$	0.0291	0.0081	11.36	7.5×10^{-7}	$< 10^{-5}$
$(400, 2)$	0.0314	0.0085	11.68	6.0×10^{-7}	$< 10^{-5}$
$(500, 2)$	0.0338	0.0090	11.88	5.1×10^{-7}	$< 10^{-5}$

experiments were performed on networks with edge density $M = 2$ and node counts $N \in \{100, 200, 300, 400, 500\}$. All ablations are conducted with the same diffusion prior.

Fig. 13 compares the robustness metric values obtained under different operator settings. The degree-balancing operator Ω consistently improves robustness over the baseline, consistent with the well-established link between degree homogeneity and enhanced tolerance to high-degree attacks. In contrast, the bridging operator ψ, when used alone, does not lead to robustness improvement. Although ψ effectively encourages cross-component linkage, it does not by itself improve robustness when the underlying degree distribution remains highly imbalanced.

When Ω and ψ are jointly applied, robustness increases substantially. This indicates that degree balancing establishes

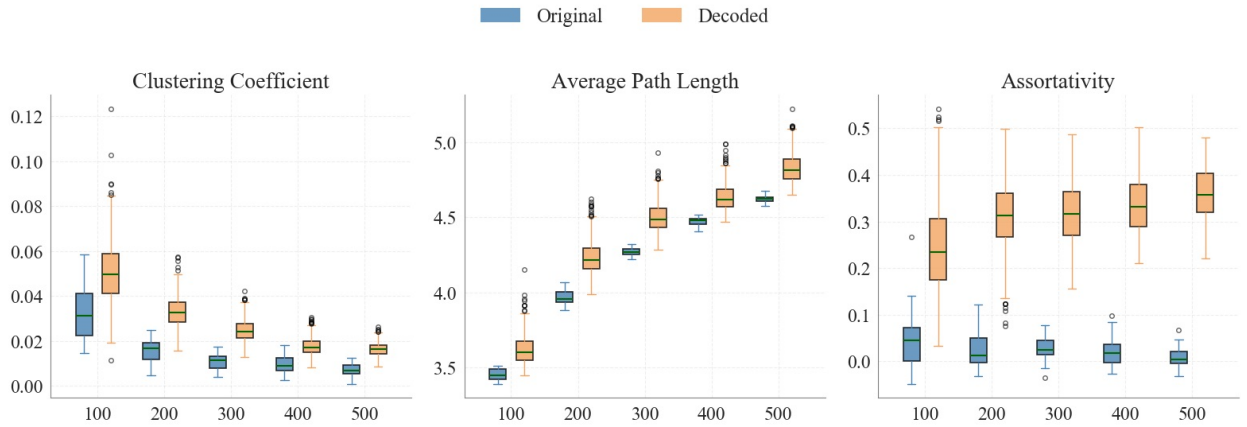


Fig. 12: Distributions of graph attributes before and after DIFF-TRO decoding ($M = 2$, $N = 100$–500).

TABLE VI: Graph attribute comparison between original and DIFF-TRO-decoded topologies (mean±std over 10 runs).

N	**Orig. Clust.**	**Dec. Clust.**	**Orig. Path**	**Dec. Path**	**Orig. Assort.**	**Dec. Assort.**
100	0.0332 ± 0.0118	0.0566 ± 0.0130	3.4554 ± 0.0384	3.6237 ± 0.1091	0.0505 ± 0.0724	0.2664 ± 0.0941
200	0.0160 ± 0.0057	0.0333 ± 0.0053	3.9674 ± 0.0488	4.2245 ± 0.0833	0.0254 ± 0.0427	0.3035 ± 0.0796
300	0.0106 ± 0.0038	0.0251 ± 0.0060	4.2708 ± 0.0266	4.4980 ± 0.0906	0.0281 ± 0.0268	0.3018 ± 0.0697
400	0.0097 ± 0.0043	0.0184 ± 0.0043	4.4735 ± 0.0270	4.6533 ± 0.0831	0.0217 ± 0.0351	0.3426 ± 0.0604
500	0.0072 ± 0.0034	0.0161 ± 0.0030	4.6200 ± 0.0257	4.8273 ± 0.1014	0.0079 ± 0.0241	0.3607 ± 0.0599

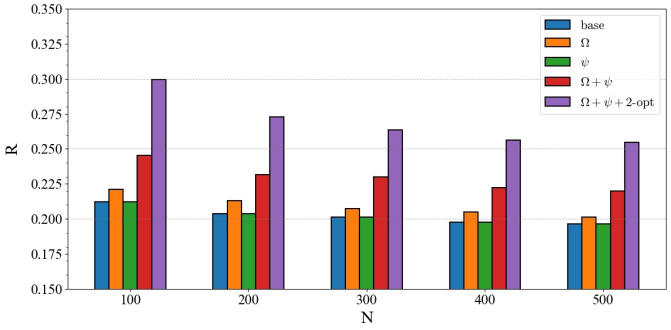


Fig. 13: Ablation study: effect of each module on robustness metric R. Mean ± std over 10 runs.

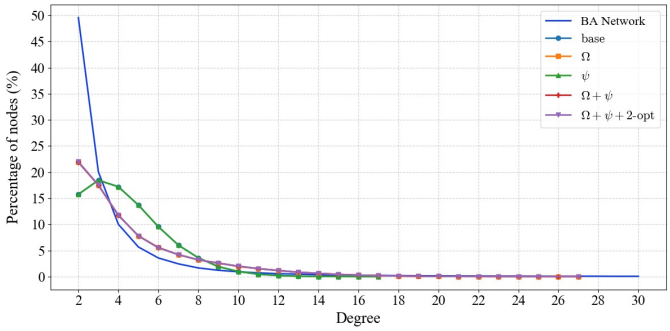


Fig. 14: Degree distributions under decoding-operator variants ($M = 2$, $N = 100$–500; mean over 10 runs).

a stable local structure that enables component bridging to produce globally more resilient connectivity patterns. Finally, incorporating a lightweight 2-opt perturbation at the end of each epoch further enhances robustness. Small random structural explorations help the decoder escape locally optimal but suboptimal configurations.

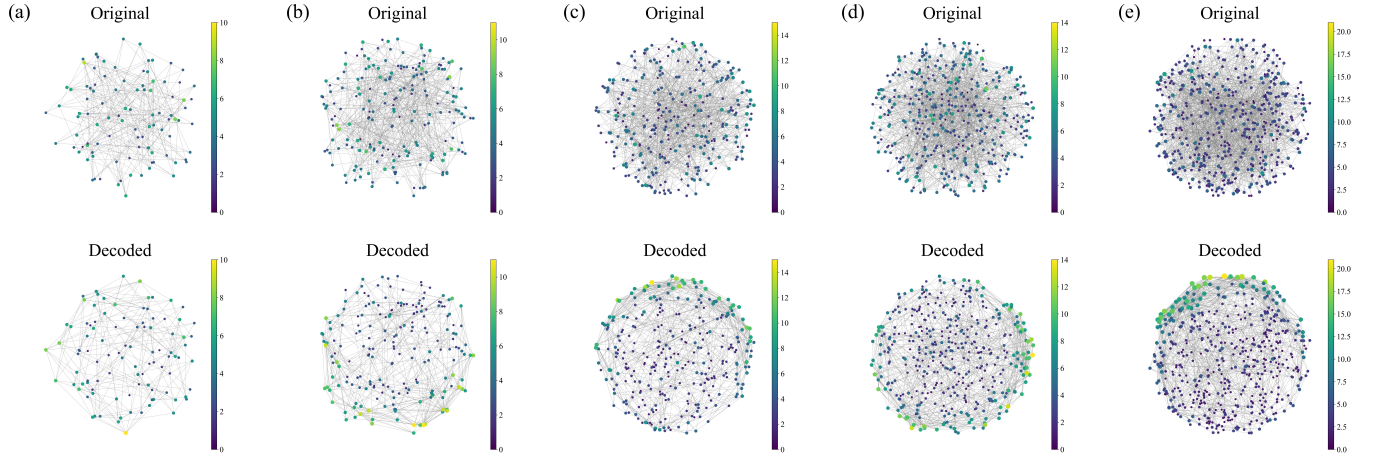
Fig. 14 further illustrates the corresponding degree distributions. Without degree balancing, the topology exhibits noticeable skewness, whereas the use of Ω (or its combinations) produces stable and well-controlled degree profiles, aligning with the observed robustness improvements.

I. Topology Visualization

To qualitatively evaluate the effect of optimization, Fig. 15 visualizes topologies before and after optimization. The top row shows initial random geometric graphs; the bottom row presents optimized structures produced by DIFF-TRO after 10 epochs. Each column corresponds to a different N value (100–500) under fixed $M = 2$. The optimized networks exhibit stronger global connectivity and reduced vulnerability to high-degree node removal.

VII. Discussion and Future Work

The proposed DIFF-TRO framework validates the feasibility of addressing path dependence and data scarcity in IoT topology robustness optimization through a diffusion-driven, self-supervised generative paradigm. By operating in a continuous latent space and learning robustness-related structural priors directly from unlabeled topologies, DIFF-TRO differs from conventional search-based and supervised approaches, whose performance is constrained by local edge swaps or limited labeled datasets. The constraint-aware decoding strategy further enables the generation of geometrically feasible and robustness-enhanced topologies under communication constraints.



(a) $N = 100$ (b) $N = 200$ (c) $N = 300$ (d) $N = 400$ (e) $N = 500$

Fig. 15: Topology visualization for $M = 2$ and $N = 100$–500. For each N: the top is an initial topology; the bottom is a decoded optimized topology (DIFF-TRO).

Despite these advantages, several limitations remain. First, diffusion-based denoising involves iterative sampling, which increases computational cost and leads to higher inference latency than reinforcement learning approaches such as ROMEM [11], particularly in large networks. Second, the current decoding module relies on explicit edge ranking and feasibility checks; although effective, it introduces additional overhead and remains partially hand-crafted. Third, DIFF-TRO focuses on static topologies, while dynamic or mobility-driven IoT environments require robustness optimization under time-varying constraints.

Future research may explore efficient diffusion variants—such as jump-step sampling, stochastic skipping, or learned denoising schedules—to accelerate inference while preserving robustness. Developing differentiable or learning-based decoding mechanisms may further reduce reconstruction overhead and better integrate domain constraints. Extending DIFF-TRO to dynamic, multi-domain, or federated IoT settings is another promising direction, potentially enabling resilient topology optimization in practical large-scale deployments.

VIII. Conclusion

This paper presented DIFF-TRO, a diffusion-based self-supervised framework that directly addresses the core limitations of existing IoT topology robustness optimization methods–namely, path dependence in search-based algorithms and data scarcity in supervised models. By reformulating topology design as a denoising diffusion process over spatio-temporal graph embeddings, DIFF-TRO learns robustness-oriented structural priors from unlabeled topologies, which reduces reliance on labeled datasets and avoids the early-commitment bias of sequential rewiring.

Complemented by a constraint-aware decoding strategy, DIFF-TRO reconstructs topologies that satisfy communication range, degree balance, and component-connectivity requirements, while the closed-loop retraining mechanism enables progressive self-improvement even under limited data availability. Extensive experiments across diverse network scales and densities demonstrate that DIFF-TRO consistently surpasses heuristic and reinforcement-learning baselines in both robustness and convergence stability.

While diffusion modeling introduces additional computational overhead, especially during iterative sampling, DIFF-TRO provides a principled and scalable foundation for topology optimization in next-generation IoT systems. Promising directions include accelerating inference with learned denoising schedules, replacing the hand-crafted decoder with a learnable one, and incorporating richer physical-layer constraints such as energy, delay, and interference. DIFF-TRO thus offers a practical basis for robustness-oriented topology design in large-scale IoT systems.

References

[1] V. Goswami, P. Jadav, and S. K. Soni, "Review on how iiot has revolutionized greenhouse, manufacturing and medical industries," in *Recent Advances in Mechanical Infrastructure: Proceedings of ICRAM 2021*. Springer, 2022, pp. 179–192.

[2] C. Dey and S. K. Sen, *Industrial automation technologies*. CRC Press, 2020.

[3] S. Muthuramalingam, A. Bharathi, S. Rakesh Kumar, N. Gayathri, R. Sathiyaraj, and B. Balamurugan, "Iot based intelligent transportation system (iot-its) for global perspective: A case study," *Internet of things and big data analytics for smart generation*, pp. 279–300, 2019.

[4] N. Y. Philip, J. J. Rodrigues, H. Wang, S. J. Fong, and J. Chen, "Internet of things for in-home health monitoring systems: Current advances, challenges and future directions," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 2, pp. 300–310, 2021.

[5] S. Selvaraj and S. Sundaravaradhan, "Challenges and opportunities in iot healthcare systems: a systematic review," *SN Applied Sciences*, vol. 2, no. 1, p. 139, 2020.

[6] S. Samuel, G. Cole, and M. J. Eacott, "Grammatical gender and linguistic relativity: A systematic review," *Psychonomic bulletin & review*, vol. 26, no. 6, pp. 1767–1786, 2019.

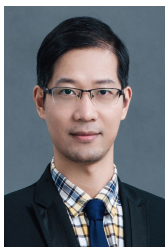
[7] Y. Yuan, L. Jiao, K. Zhu, X. Lin, and L. Zhang, "Ai in 5g: The case of online distributed transfer learning over edge networks," in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2022, pp. 810–819.

[8] R. Yu, G. Xue, and X. Zhang, "Application provisioning in fog computing-enabled internet-of-things: A network perspective," in *IEEE INFOCOM 2018-IEEE Conference on Computer Communications*. IEEE, 2018, pp. 783–791.

[9] T. Limbasiya, D. Das, and S. K. Das, "Mcomiov: Secure and energy-efficient message communication protocols for internet of vehicles," *IEEE/ACM Transactions On Networking*, vol. 29, no. 3, pp. 1349–1361, 2021.
[10] S. Amar, D. Chisnall, T. Chen, N. W. Filardo, B. Laurie, K. Liu, R. Norton, S. W. Moore, Y. Tao, R. N. Watson, *et al.*, "Cheriot: Complete memory safety for embedded devices," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 641–653.
[11] T. Qiu, N. Chen, and S. Zhang, *Robustness Optimization for IoT Topology*. Springer, no. PUBDB-2022-03827.
[12] H. J. Herrmann, C. M. Schneider, A. A. Moreira, J. S. Andrade, and S. Havlin, "Onion-like network topology enhances robustness against malicious attacks," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2011, no. 01, p. P01027, 2011.
[13] T. Qiu, J. Liu, W. Si, and D. O. Wu, "Robustness optimization scheme with multi-population co-evolution for scale-free wireless sensor networks," *IEEE/ACM Transactions on Networking*, vol. 27, no. 3, pp. 1028–1042, 2019.
[14] Y. Peng, C. Liu, S. Liu, Y. Liu, and Y. Wu, "Smarttro: Optimizing topology robustness for internet of things via deep reinforcement learning with graph convolutional networks," *Computer Networks*, vol. 218, p. 109385, 2022.
[15] T. Qiu, X. Yang, N. Chen, S. Zhang, G. Min, and D. O. Wu, "A self-adaptive robustness optimization method with evolutionary multi-agent for iot topology," *IEEE/ACM Transactions on Networking*, 2023.
[16] N. Chen, T. Qiu, C. Mu, M. Han, and P. Zhou, "Deep actor–critic learning-based robustness enhancement of internet of things," *IEEE Internet of Things Journal*, vol. 7, no. 7, pp. 6191–6200, 2020.
[17] I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio, "Neural combinatorial optimization with reinforcement learning," *arXiv preprint arXiv:1611.09940*, 2016.
[18] N. Chen, T. Qiu, X. Zhou, K. Li, and M. Atiquzzaman, "An intelligent robust networking mechanism for the internet of things," *IEEE Communications Magazine*, vol. 57, no. 11, pp. 91–95, 2019.
[19] X. Ying, "An overview of overfitting and its solutions," in *Journal of physics: Conference series*, vol. 1168. IOP Publishing, 2019, p. 022022.
[20] C. M. Schneider, A. A. Moreira, J. S. Andrade Jr, S. Havlin, and H. J. Herrmann, "Mitigation of malicious attacks on networks," *Proceedings of the National Academy of Sciences*, vol. 108, no. 10, pp. 3838–3841, 2011.
[21] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.
[22] Y. Song and S. Ermon, "Generative modeling by estimating gradients of the data distribution," *Advances in neural information processing systems*, vol. 32, 2019.
[23] L. Yang, Z. Zhang, Y. Song, S. Hong, R. Xu, Y. Zhao, W. Zhang, B. Cui, and M.-H. Yang, "Diffusion models: A comprehensive survey of methods and applications," *ACM Computing Surveys*, vol. 56, no. 4, pp. 1–39, 2023.
[24] A. Q. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," in *International conference on machine learning*. PMLR, 2021, pp. 8162–8171.
[25] J. Hu, X. Liu, Z. Fan, Y. Liang, and R. Zimmermann, "Towards unifying diffusion models for probabilistic spatio-temporal graph learning," in *Proceedings of the 32nd ACM International Conference on Advances in Geographic Information Systems*, 2024, pp. 135–146.
[26] Z. Sun and Y. Yang, "Difusco: Graph-based diffusion solvers for combinatorial optimization," *Advances in Neural Information Processing Systems*, vol. 36, pp. 3706–3731, 2023.
[27] K. Yu, H. Zhao, Y. Huang, R. Yi, K. Xu, and C. Zhu, "Disco: Efficient diffusion solver for large-scale combinatorial optimization problems," *arXiv preprint arXiv:2406.19705*, 2024.
[28] Y. Li, J. Guo, R. Wang, H. Zha, and J. Yan, "Fast t2t: Optimization consistency speeds up diffusion-based training-to-testing solving for combinatorial optimization," *Advances in Neural Information Processing Systems*, vol. 37, pp. 30 179–30 206, 2024.
[29] Y. Li, L. Chen, H. Wang, R. Wang, and J. Yan, "Generation as search operator for test-time scaling of diffusion-based combinatorial optimization," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
[30] X. Bresson and T. Laurent, "An experimental study of neural networks for variable graphs," 2018.
[31] T. Tanizawa, "Structural robustness and transport efficiency of complex networks with degree correlation," *Nonlinear Theory and Its Applications, IEICE*, vol. 4, no. 2, pp. 138–147, 2013.
[32] M. Murakami, S. Ishikura, D. Kominami, T. Shimokawa, and M. Murata, "Robustness and efficiency in interconnected networks with changes in network assortativity," *Applied Network Science*, vol. 2, no. 1, p. 6, 2017.
[33] T. Qiu, L. Zhang, N. Chen, S. Zhang, W. Liu, and D. O. Wu, "Born this way: A self-organizing evolution scheme with motif for internet of things robustness," *IEEE/ACM Transactions on Networking*, vol. 30, no. 6, pp. 2644–2657, 2022.
[34] T. Qiu, J. Sun, N. Chen, S. Zhang, W. Si, and X. Wang, "Olive-like networking: A uniformity driven robust topology generation scheme for iot system," *IEEE Transactions on Computers*, 2024.
[35] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

Jiqiao Li (Student Member, IEEE) received the B.S. degree in software engineering from the Dalian University of Technology (DUT) in 2023. He is currently pursuing the M.E. degree in the School of Computer Science and Technology, Tianjin University, China. His research interests include the Internet of Things and robustness optimization. He is a student member of the Association for Computing Machinery (ACM). He has received scholarships such as Hisense Scholarship and several awards including Outstanding Graduate of Liaoning Province.

Ning Chen (S'19-M'24) is a postdoc at the Department of Computer Science, City University of Hong Kong. He received his PhD at the School of Computer Science and Technology, Tianjin University, China. He received his B.E. and M.E. separately in software engineering from the Dalian University of Technology. He was an exchange student at the Institute of Information of the University of Amsterdam in the Netherlands (2021-2022), where CSC fully supported the scholarship. He has served as the organizing committee as well as Program Committee (PC) members for 5 important international conferences and reviewers for 9 prestigious journals, including IEEE/ACM ToN, IEEE TMC, etc. He has co-authored 1 book and over 20 scientific papers in international journals and conference proceedings, such as IEEE/ACM ToN, IEEE TMC, etc. There are 2 papers listed as ESI highly cited papers. He has won several awards, such as National Scholarship, Outstanding Graduate in Liaoning Province, Excellent Student Cadre, etc. His research interests cover the Internet of Things, complex networks, and topology robustness optimization. He is a Member of the China Computer Federation (CCF), IEEE, and ACM.



Xiaobo Zhou (Senior Member, IEEE) received the BSc degree in electronic information science and technology from the University of Science and Technology of China, Hefei, China, in 2007, the ME degree in computer application technology from the Graduate University of Chinese Academy of Science, Beijing, China, in 2010, and the PhD degree in information science from the School of Information Science, Japan Advanced Institute of Science and Technology, Nomi, Ishikawa, Japan, in 2013. From 2014 to 2015, he was a researcher with the Department of Communications Engineering, University of Oulu, Oulu, Finland. He is currently a professor with the School of Computer Science and Technology, College of Intelligence and Computing, Tianjin University, Tianjin, China. His research interests include Internet of Things, Cloud Computing, Data Center Networks, Vehicular Networks, and Mobile-Edge Computing.

Tie Qiu (Senior Member, IEEE) is currently a full professor with the School of Computer Science and Engineering, Northeastern University, China. Prior to this position, he held professor in 2018 with the School of Computer Science and Technology, Tianjin University, an associate professor, in 2013 and an assistant professor, in 2008 with the School of Software, Dalian University of Technology. He was a visiting professor with the Department of Electrical and Computer Engineering, Iowa State University, USA, from 2014 to 2015. He has contributed to the development of 11 copyrighted software systems and invented 40 patents. There are 21 papers listed as ESI highly cited papers. He has authored/coauthored more than 10 books, and more than 200 scientific papers in international journals and conference proceedings, such as IEEE/ACM Transactions on Networking (ToN), IEEE Transactions on Mobile Computing (TMC), IEEE Transactions on Knowledge and Data Engineering (TKDE), IEEE Transactions on Industrial Informatics, IEEE Communications Surveys & Tutorials, IEEE Communications Magazine, INFOCOM, and GLOBECOM. He serves as an associate editor for IEEE/ACM Transactions on Networking (ToN), and an area editor for Ad Hoc Networks (Elsevier). He serves as the general chair, program chair, workshop chair, publicity chair, publication chair, or a TPC Member for a number of international conferences. He is a distinguished member of the China Computer Federation (CCF) and a Senior Member of ACM.